```
create trigger overdraft-trigger on account
for update
as
if nrow.balance < 0
begin
   insert into borrower
          (select customer-name, account-number
           from depositor, inserted
           where inserted.account-number = depositor.account-number)
    insert into loan values
          (inserted.account-number, inserted.branch-name, − inserted.balance)
   update account set balance = 0
          from account, inserted
          where account.account-number = inserted.account-number
end
```

Figure 6.5 Example of trigger in MS-SQL server syntax

easier way to maintain summary data. Designers also used triggers extensively for replicating databases; they used triggers on insert/delete/update of each relation to record the changes in relations called **change** or **delta** relations. A separate process copied over the changes to the replica (copy) of the database, and the system executed the changes on the replica. Modern database systems, however, provide built-in facilities for database replication, making triggers unnecessary for replication in most cases.

In fact, many trigger applications, including our example overdraft trigger, can be substituted by "encapsulation" features being introduced in SQL:1999. Encapsulation can be used to ensure that updates to the *balance* attribute of *account* are done only through a special procedure. That procedure would in turn check for negative balance, and carry out the actions of the overdraft trigger. Encapsulations can replace the reorder trigger in a similar manner.

Triggers should be written with great care, since a trigger error detected at run time causes the failure of the insert/delete/update statement that set off the trigger. Furthermore, the action of one trigger can set off another trigger. In the worst case, this could even lead to an infinite chain of triggering. For example, suppose an insert trigger on a relation has an action that causes another (new) insert on the same relation. The insert action then triggers yet another insert action, and so on ad infinitum. Database systems typically limit the length of such chains of triggers (for example to 16 or 32), and consider longer chains of triggering an error.

Triggers are occasionally called *rules*, or *active rules*, but should not be confused with Datalog rules (see Section 5.2), which are really view definitions.

6.5 Security and Authorization

The data stored in the database need protection from unauthorized access and malicious destruction or alteration, in addition to the protection against accidental intro-

duction of inconsistency that integrity constraints provide. In this section, we examine the ways in which data may be misused or intentionally made inconsistent. We then present mechanisms to guard against such occurrences.

6.5.1 Security Violations

Among the forms of malicious access are:

- Unauthorized reading of data (theft of information)
- Unauthorized modification of data
- Unauthorized destruction of data

Database security refers to protection from malicious access. Absolute protection of the database from malicious abuse is not possible, but the cost to the perpetrator can be made high enough to deter most if not all attempts to access the database without proper authority.

To protect the database, we must take security measures at several levels:

- **Database system**. Some database-system users may be authorized to access only a limited portion of the database. Other users may be allowed to issue queries, but may be forbidden to modify the data. It is the responsibility of the database system to ensure that these authorization restrictions are not violated.
- **Operating system**. No matter how secure the database system is, weakness in operating-system security may serve as a means of unauthorized access to the database.
- **Network**. Since almost all database systems allow remote access through terminals or networks, software-level security within the network software is as important as physical security, both on the Internet and in private networks.
- **Physical**. Sites with computer systems must be physically secured against armed or surreptitious entry by intruders.
- **Human**. Users must be authorized carefully to reduce the chance of any user giving access to an intruder in exchange for a bribe or other favors.

Security at all these levels must be maintained if database security is to be ensured. A weakness at a low level of security (physical or human) allows circumvention of strict high-level (database) security measures.

In the remainder of this section, we shall address security at the database-system level. Security at the physical and human levels, although important, is beyond the scope of this text.

Security within the operating system is implemented at several levels, ranging from passwords for access to the system to the isolation of concurrent processes running within the system. The file system also provides some degree of protection. The

bibliographical notes reference coverage of these topics in operating-system texts. Finally, network-level security has gained widespread recognition as the Internet has evolved from an academic research platform to the basis of international electronic commerce. The bibliographic notes list textbook coverage of the basic principles of network security. We shall present our discussion of security in terms of the relational-data model, although the concepts of this chapter are equally applicable to all data models.

6.5.2 Authorization

We may assign a user several forms of authorization on parts of the database. For example,

- **Read authorization** allows reading, but not modification, of data.
- **Insert authorization** allows insertion of new data, but not modification of existing data.
- **Update authorization** allows modification, but not deletion, of data.
- **Delete authorization** allows deletion of data.

We may assign the user all, none, or a combination of these types of authorization.

In addition to these forms of authorization for access to data, we may grant a user authorization to modify the database schema:

- **Index authorization** allows the creation and deletion of indices.
- **Resource authorization** allows the creation of new relations.
- **Alteration authorization** allows the addition or deletion of attributes in a relation.
- **Drop authorization** allows the deletion of relations.

The **drop** and **delete** authorization differ in that **delete** authorization allows deletion of tuples only. If a user deletes all tuples of a relation, the relation still exists, but it is empty. If a relation is dropped, it no longer exists.

We regulate the ability to create new relations through **resource** authorization. A user with **resource** authorization who creates a new relation is given all privileges on that relation automatically.

Index authorization may appear unnecessary, since the creation or deletion of an index does not alter data in relations. Rather, indices are a structure for performance enhancements. However, indices also consume space, and all database modifications are required to update indices. If **index** authorization were granted to all users, those who performed updates would be tempted to delete indices, whereas those who issued queries would be tempted to create numerous indices. To allow the *database administrator* to regulate the use of system resources, it is necessary to treat index creation as a privilege.

The ultimate form of authority is that given to the database administrator. The database administrator may authorize new users, restructure the database, and so on. This form of authorization is analogous to that of a **superuser** or operator for an operating system.

6.5.3 Authorization and Views

In Chapter 3, we introduced the concept of *views* as a means of providing a user with a personalized model of the database. A view can hide data that a user does not need to see. The ability of views to hide data serves both to simplify usage of the system and to enhance security. Views simplify system usage because they restrict the user's attention to the data of interest. Although a user may be denied direct access to a relation, that user may be allowed to access part of that relation through a view. Thus, a combination of relational-level security and view-level security limits a user's access to precisely the data that the user needs.

In our banking example, consider a clerk who needs to know the names of all customers who have a loan at each branch. This clerk is not authorized to see information regarding specific loans that the customer may have. Thus, the clerk must be denied direct access to the *loan* relation. But, if she is to have access to the information needed, the clerk must be granted access to the view *cust-loan*, which consists of only the names of customers and the branches at which they have a loan. This view can be defined in SQL as follows:

```
create view cust-loan as
    (select branch-name, customer-name
     from borrower, loan
     where borrower.loan-number = loan.loan-number)
```

Suppose that the clerk issues the following SQL query:

```
select *
from cust-loan
```

Clearly, the clerk is authorized to see the result of this query. However, when the query processor translates it into a query on the actual relations in the database, it produces a query on *borrower* and *loan*. Thus, the system must check authorization on the clerk's query before it begins query processing.

Creation of a view does not require **resource** authorization. A user who creates a view does not necessarily receive all privileges on that view. She receives only those privileges that provide no additional authorization beyond those that she already had. For example, a user cannot be given **update** authorization on a view without having **update** authorization on the relations used to define the view. If a user creates a view on which no authorization can be granted, the system will deny the view creation request. In our *cust-loan* view example, the creator of the view must have **read** authorization on both the *borrower* and *loan* relations.

6.5.4 Granting of Privileges

A user who has been granted some form of authorization may be allowed to pass on this authorization to other users. However, we must be careful how authorization may be passed among users, to ensure that such authorization can be revoked at some future time.

Consider, as an example, the granting of update authorization on the *loan* relation of the bank database. Assume that, initially, the database administrator grants update authorization on *loan* to users U_1, U_2, and U_3, who may in turn pass on this authorization to other users. The passing of authorization from one user to another can be represented by an **authorization graph**. The nodes of this graph are the users. The graph includes an edge $U_i \rightarrow U_j$ if user U_i grants update authorization on *loan* to U_j. The root of the graph is the database administrator. In the sample graph in Figure 6.6, observe that user U_5 is granted authorization by both U_1 and U_2; U_4 is granted authorization by only U_1.

A user has an authorization *if and only if* there is a path from the root of the authorization graph (namely, the node representing the database administrator) down to the node representing the user.

Suppose that the database administrator decides to revoke the authorization of user U_1. Since U_4 has authorization from U_1, that authorization should be revoked as well. However, U_5 was granted authorization by both U_1 and U_2. Since the database administrator did not revoke update authorization on *loan* from U_2, U_5 retains update authorization on *loan*. If U_2 eventually revokes authorization from U_5, then U_5 loses the authorization.

A pair of devious users might attempt to defeat the rules for revocation of authorization by granting authorization to each other, as shown in Figure 6.7a. If the database administrator revokes authorization from U_2, U_2 retains authorization through U_3, as in Figure 6.7b. If authorization is revoked subsequently from U_3, U_3 appears to retain authorization through U_2, as in Figure 6.7c. However, when the database administrator revokes authorization from U_3, the edges from U_3 to U_2 and from U_2 to U_3 are no longer part of a path starting with the database administrator.

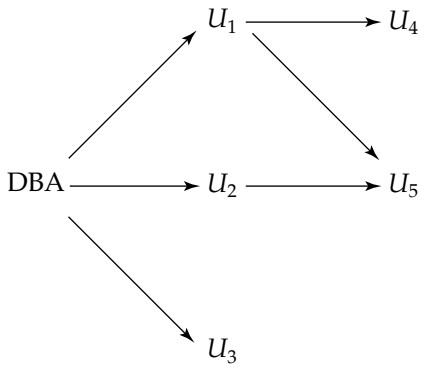


Figure 6.6 Authorization-grant graph.

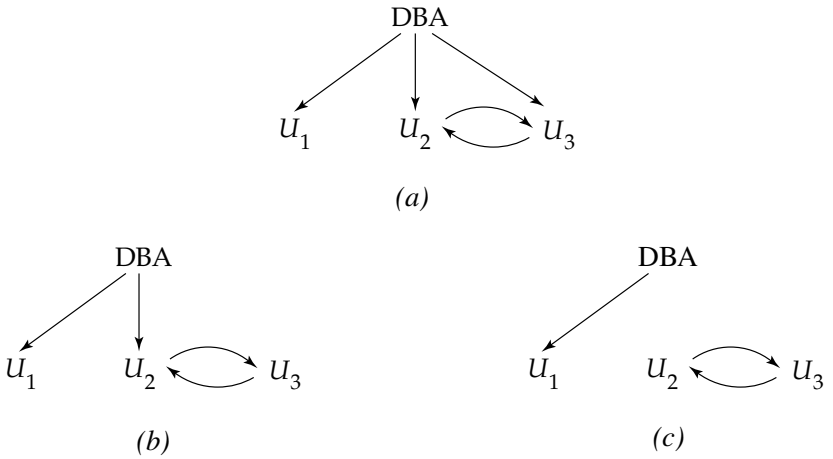


Figure 6.7 Attempt to defeat authorization revocation.

We require that all edges in an authorization graph be part of some path originating with the database administrator. The edges between U_2 and U_3 are deleted, and the resulting authorization graph is as in Figure 6.8.

6.5.5 Notion of Roles

Consider a bank where there are many tellers. Each teller must have the same types of authorizations to the same set of relations. Whenever a new teller is appointed, she will have to be given all these authorizations individually.

A better scheme would be to specify the authorizations that every teller is to be given, and to separately identify which database users are tellers. The system can use these two pieces of information to determine the authorizations of each person who is a teller. When a new person is hired as a teller, a user identifier must be allocated to him, and he must be identified as a teller. Individual permissions given to tellers need not be specified again.

The notion of **roles** captures this scheme. A set of roles is created in the database. Authorizations can be granted to roles, in exactly the same fashion as they are granted to individual users. Each database user is granted a set of roles (which may be empty) that he or she is authorized to perform.

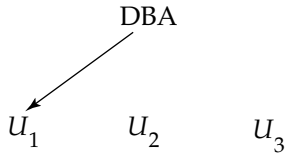


Figure 6.8 Authorization graph.

In our bank database, examples of roles could include *teller*, *branch-manager*, *auditor*, and *system-administrator*.

A less preferable alternative would be to create a *teller* userid, and permit each teller to connect to the database using the *teller* userid. The problem with this scheme is that it would not be possible to identify exactly which teller carried out a transaction, leading to security risks. The use of roles has the benefit of requiring users to connect to the database with their own userid.

Any authorization that can be granted to a user can be granted to a role. Roles are granted to users just as authorizations are. And like other authorizations, a user may also be granted authorization to grant a particular role to others. Thus, branch managers may be granted authorization to grant the *teller* role.

6.5.6 Audit Trails

Many secure database applications require an **audit trail** be maintained. An audit trail is a log of all changes (inserts/deletes/updates) to the database, along with information such as which user performed the change and when the change was performed.

The audit trail aids security in several ways. For instance, if the balance on an account is found to be incorrect, the bank may wish to trace all the updates performed on the account, to find out incorrect (or fraudulent) updates, as well as the persons who carried out the updates. The bank could then also use the audit trail to trace all the updates performed by these persons, in order to find other incorrect or fraudulent updates.

It is possible to create an audit trail by defining appropriate triggers on relation updates (using system-defined variables that identify the user name and time). However, many database systems provide built-in mechanisms to create audit trails, which are much more convenient to use. Details of how to create audit trails vary across database systems, and you should refer the database system manuals for details.

6.6 Authorization in SQL

The SQL language offers a fairly powerful mechanism for defining authorizations. We describe these mechanisms, as well as their limitations, in this section.

6.6.1 Privileges in SQL

The SQL standard includes the privileges **delete**, **insert**, **select**, and **update**. The **select** privilege corresponds to the **read** privilege. SQL also includes a **references** privilege that permits a user/role to declare foreign keys when creating relations. If the relation to be created includes a foreign key that references attributes of another relation, the user/role must have been granted **references** privilege on those attributes. The reason that the **references** privilege is a useful feature is somewhat subtle; we explain the reason later in this section.

The SQL data-definition language includes commands to grant and revoke privileges. The **grant** statement is used to confer authorization. The basic form of this statement is:

grant <privilege list> **on** <relation name or view name> **to** <user/role list>

The *privilege list* allows the granting of several privileges in one command.

The following **grant** statement grants users U_1, U_2, and U_3 **select** authorization on the *account* relation:

grant select on *account* **to** U_1, U_2, U_3

The **update** authorization may be given either on all attributes of the relation or on only some. If **update** authorization is included in a **grant** statement, the list of attributes on which update authorization is to be granted optionally appears in parentheses immediately after the **update** keyword. If the list of attributes is omitted, the update privilege will be granted on all attributes of the relation.

This **grant** statement gives users U_1, U_2, and U_3 update authorization on the *amount* attribute of the *loan* relation:

grant update (*amount*) **on** *loan* **to** U_1, U_2, U_3

The **insert** privilege may also specify a list of attributes; any inserts to the relation must specify only these attributes, and the system either gives each of the remaining attributes default values (if a default is defined for the attribute) or sets them to null.

The SQL **references** privilege is granted on specific attributes in a manner like that for the **update** privilege. The following **grant** statement allows user U_1 to create relations that reference the key *branch-name* of the *branch* relation as a foreign key:

grant references (*branch-name*) **on** *branch* **to** U_1

Initially, it may appear that there is no reason ever to prevent users from creating foreign keys referencing another relation. However, recall from Section 6.2 that foreign-key constraints restrict deletion and update operations on the referenced relation. In the preceding example, if U_1 creates a foreign key in a relation r referencing the *branch-name* attribute of the *branch* relation, and then inserts a tuple into r pertaining to the Perryridge branch, it is no longer possible to delete the Perryridge branch from the *branch* relation without also modifying relation r. Thus, the definition of a foreign key by U_1 restricts future activity by other users; therefore, there is a need for the **references** privilege.

The privilege **all privileges** can be used as a short form for all the allowable privileges. Similarly, the user name **public** refers to all current and future users of the system. SQL also includes a **usage** privilege that authorizes a user to use a specified domain (recall that a domain corresponds to the programming-language notion of a type, and may be user defined).

6.6.2 Roles

Roles can be created in SQL:1999 as follows

create role *teller*

Roles can then be granted privileges just as the users can, as illustrated in this statement:

grant select on *account*
to *teller*

Roles can be asigned to the users, as well as to some other roles, as these statements show.

grant *teller* **to** john
create role *manager*
grant *teller* **to** *manager*
grant *manager* **to** mary

Thus the privileges of a user or a role consist of

- All privileges directly granted to the user/role
- All privileges granted to roles that have been granted to the user/role

Note that there can be a chain of roles; for example, the role *employee* may be granted to all *teller*s. In turn the role *teller* is granted to all *manager*s. Thus, the *manager* role inherits all privileges granted to the roles *employee* and to *teller* in addition to privileges granted directly to *manager*.

6.6.3 The Privilege to Grant Privileges

By default, a user/role that is granted a privilege is not authorized to grant that privilege to another user/role. If we wish to grant a privilege and to allow the recipient to pass the privilege on to other users, we append the **with grant option** clause to the appropriate **grant** command. For example, if we wish to allow U_1 the **select** privilege on *branch* and allow U_1 to grant this privilege to others, we write

grant select on *branch* **to** U_1 **with grant option**

To revoke an authorization, we use the **revoke** statement. It takes a form almost identical to that of **grant**:

revoke <privilege list> **on** <relation name or view name>
from <user/role list> [**restrict** | **cascade**]

Thus, to revoke the privileges that we granted previously, we write

revoke select on *branch* **from** U_1, U_2, U_3
revoke update (*amount*) **on** *loan* **from** U_1, U_2, U_3
revoke references (*branch-name*) **on** *branch* **from** U_1

As we saw in Section 6.5.4, the revocation of a privilege from a user/role may cause other users/roles also to lose that privilege. This behavior is called *cascading of the revoke*. In most database systems, cascading is the default behavior; the keyword **cascade** can thus be omitted, as we have done in the preceding examples. The **revoke** statement may alternatively specify **restrict**:

revoke select on *branch* **from** U_1, U_2, U_3 **restrict**

In this case, the system returns an error if there are any cascading revokes, and does not carry out the revoke action. The following **revoke** statement revokes only the grant option, rather than the actual **select** privilege:

revoke grant option for select on *branch* **from** U_1

6.6.4 Other Features

The creator of an object (relation/view/role) gets all privileges on the object, including the privilege to grant privileges to others.

The SQL standard specifies a primitive authorization mechanism for the database schema: Only the owner of the schema can carry out any modification to the schema. Thus, schema modifications—such as creating or deleting relations, adding or dropping attributes of relations, and adding or dropping indices—may be executed by only the owner of the schema. Several database implementations have more powerful authorization mechanisms for database schemas, similar to those discussed earlier, but these mechanisms are nonstandard.

6.6.5 Limitations of SQL Authorization

The current SQL standards for authorization have some shortcomings. For instance, suppose you want all students to be able to see their own grades, but not the grades of anyone else. Authorization must then be at the level of individual tuples, which is not possible in the SQL standards for authorization.

Furthermore, with the growth in the Web, database accesses come primarily from Web application servers. The end users may not have individual user identifiers on the database, and indeed there may only be a single user identifier in the database corresponding to all users of an application server.

The task of authorization then falls on the application server; the entire authorization scheme of SQL is bypassed. The benefit is that fine-grained authorizations, such as those to individual tuples, can be implemented by the application. The problems are these:

- The code for checking authorization becomes intermixed with the rest of the application code.

- Implementing authorization through application code, rather than specifying it declaratively in SQL, makes it hard to ensure the absence of loopholes. Because of an oversight, one of the application programs may not check for authorization, allowing unauthorized users access to confidential data. Verifying that all application programs make all required authorization checks involves reading through all the application server code, a formidable task in a large system.

6.7 Encryption and Authentication

The various provisions that a database system may make for authorization may still not provide sufficient protection for highly sensitive data. In such cases, data may be stored in **encrypt**ed form. It is not possible for encrypted data to be read unless the reader knows how to decipher (**decrypt**) them. Encryption also forms the basis of good schemes for authenticating users to a database.

6.7.1 Encryption Techniques

There are a vast number of techniques for the encryption of data. Simple encryption techniques may not provide adequate security, since it may be easy for an unauthorized user to break the code. As an example of a weak encryption technique, consider the substitution of each character with the next character in the alphabet. Thus,

Perryridge

becomes

Qfsszsjehf

If an unauthorized user sees only "Qfsszsjehf," she probably has insufficient information to break the code. However, if the intruder sees a large number of encrypted branch names, she could use statistical data regarding the relative frequency of characters to guess what substitution is being made (for example, *E* is the most common letter in English text, followed by *T, A, O, N, I* and so on).

A good encryption technique has the following properties:

- It is relatively simple for authorized users to encrypt and decrypt data.
- It depends not on the secrecy of the algorithm, but rather on a parameter of the algorithm called the *encryption key*.
- Its encryption key is extremely difficult for an intruder to determine.

One approach, the *Data Encryption Standard* (DES), issued in 1977, does both a substitution of characters and a rearrangement of their order on the basis of an encryption key. For this scheme to work, the authorized users must be provided with the encryption key via a secure mechanism. This requirement is a major weakness, since the scheme is no more secure than the security of the mechanism by which the encryption key is transmitted. The DES standard was reaffirmed in 1983, 1987,

and again in 1993. However, weakness in DES was recongnized in 1993 as reaching a point where a new standard to be called the **Advanced Encryption Standard** (AES), needed to be selected. In 2000, the **Rijndael algorithm** (named for the inventors V. Rijmen and J. Daemen), was selected to be the AES. The Rijndael algorithm was chosen for its significantly stronger level of security and its relative ease of implementation on current computer systems as well as such devices as smart cards. Like the DES standard, the Rijndael algorithm is a shared-key (or, symmetric key) algorithm in which the authorized users share a key.

Public-key encryption is an alternative scheme that avoids some of the problems that we face with the DES. It is based on two keys; a *public key* and a *private key*. Each user U_i has a public key E_i and a private key D_i. All public keys are published: They can be seen by anyone. Each private key is known to only the one user to whom the key belongs. If user U_1 wants to store encrypted data, U_1 encrypts them using public key E_1. Decryption requires the private key D_1.

Because the encryption key for each user is public, it is possible to exchange information securely by this scheme. If user U_1 wants to share data with U_2, U_1 encrypts the data using E_2, the public key of U_2. Since only user U_2 knows how to decrypt the data, information is transferred securely.

For public-key encryption to work, there must be a scheme for encryption that can be made public without making it easy for people to figure out the scheme for decryption. In other words, it must be hard to deduce the private key, given the public key. Such a scheme does exist and is based on these conditions:

- There is an efficient algorithm for testing whether or not a number is prime.
- No efficient algorithm is known for finding the prime factors of a number.

For purposes of this scheme, data are treated as a collection of integers. We create a public key by computing the product of two large prime numbers: P_1 and P_2. The private key consists of the pair (P_1, P_2). The decryption algorithm cannot be used successfully if only the product P_1P_2 is known; it needs the individual values P_1 and P_2. Since all that is published is the product P_1P_2, an unauthorized user would need to be able to factor P_1P_2 to steal data. By choosing P_1 and P_2 to be sufficiently large (over 100 digits), we can make the cost of factoring P_1P_2 prohibitively high (on the order of years of computation time, on even the fastest computers).

The details of public-key encryption and the mathematical justification of this technique's properties are referenced in the bibliographic notes.

Although public-key encryption by this scheme is secure, it is also computationally expensive. A hybrid scheme used for secure communication is as follows: DES keys are exchanged via a public-key–encryption scheme, and DES encryption is used on the data transmitted subsequently.

6.7.2 Authentication

Authentication refers to the task of verifying the identity of a person/software connecting to a database. The simplest form of authentication consists of a secret password which must be presented when a connection is opened to a database.